

Installation

Contents

- Linux/Mac
- Windows 10+ (WSL)
- Windows (native, for use with MATLAB)
- Notes for MacOS
- Deprecated: **Make** build system

Linux/Mac

Installation on Linux, Mac and Linux within Windows WSL is described here. Additional notes on [MacOS installation](#)

Prerequisites

We assume you have: `git`, `make`, `cmake` installed on your system.

Clone acados

Clone acados and its submodules by running:

```
git clone https://github.com/acados/acados.git
cd acados
git submodule update --recursive --init
```

Installation via CMake

Install `acados` as follows:

```
mkdir -p build
cd build
cmake -DACADOS_WITH_QPOASES=ON ..
# add more optional arguments e.g. -DACADOS_WITH_DAQP=ON, a list of CMake options is provided below
make install -j4
```

Notes:

- If you get an error: `Specify a source directory [...]`, you probably forgot the `..` part in the `cmake` command
- An alternative [make build system has been deprecated](#)

CMake options:

Below is a list of CMake options available for configuring the `acados` build. These options can be passed to the `cmake` command using the `-D` flag, e.g., `cmake -DOPTION_NAME=VALUE ...`. Adjust these options based on your requirements.

Option Name	Description	Default Value
ACADOS_WITH_QPOASES	Compile acados with optional QP solver qpOASES	OFF
ACADOS_WITH_DAQP	Compile acados with optional QP solver DAQP	OFF
ACADOS_WITH_QPDUNES	Compile acados with optional QP solver qpDUNES	OFF
ACADOS_WITH_OSQP	Compile acados with optional QP solver OSQP	OFF
ACADOS_WITH_HPMPC	Compile acados with optional QP solver HPMPC	OFF
ACADOS_WITH_CLARABEL	Compile acados with optional QP solver Clarabel	OFF
ACADOS_WITH_QORE	Compile acados with optional QP solver QORE (experimental)	OFF
ACADOS_WITH_OOQP	Compile acados with optional QP solver OOQP (experimental)	OFF
BLASFEO_TARGET	BLASFEO Target architecture, see BLASFEO repository for more information. Possible values include: X64_AUTOMATIC, GENERIC, X64_INTEL_SKYLAKE_X, X64_INTEL_HASWELL, X64_INTEL_SANDY_BRIDGE, X64_INTEL_CORE, X64_AMD_BULLDOZER, ARMV8A_APPLE_M1, ARMV8A_ARM_CORTEX_A76, ARMV8A_ARM_CORTEX_A73, ARMV8A_ARM_CORTEX_A57, ARMV8A_ARM_CORTEX_A55, ARMV8A_ARM_CORTEX_A53, ARMV7A_ARM_CORTEX_A15, ARMV7A_ARM_CORTEX_A9, ARMV7A_ARM_CORTEX_A7	X64_AUTOMATIC
LA	Linear algebra optimization level for BLASFEO	HIGH_PERFORMANCE
ACADOS_WITH_SYSTEM_BLASFEO	Use BLASFEO found via <code>find_package(blasfeo)</code> instead of compiling it	OFF
HPIPM_TARGET	HPIPM Target architecture. Possible values: AVX, GENERIC	GENERIC
ACADOS_WITH_OPENMP	OpenMP parallelization	OFF
ACADOS_NUM_THREADS	Number of threads for OpenMP parallelization within one NLP solver. If not set, <code>omp_get_max_threads</code> will be used to determine the number of threads. If multiple solves should be parallelized, e.g. with an <code>AcadosOcpBatchSolver</code> or <code>AcadosSimBatchSolver</code> , set this to 1.	Not set
ACADOS_SILENT	No console status output	OFF
ACADOS_DEBUG_SQP_PRINT_QPS_TO_FILE	Print QP inputs and outputs to file in SQP	OFF
ACADOS_DEVELOPER_DEBUG_CHECKS	Enable developer debug checks	OFF
CMAKE_BUILD_TYPE	Build type (e.g., Release, Debug, etc.)	Release
ACADOS_UNIT_TESTS	Compile unit tests	OFF
ACADOS_EXAMPLES	Compile C examples	OFF
ACADOS_OCTAVE	Octave interface CMake tests	OFF
BUILD_SHARED_LIBS	Build shared libraries, should be <code>ON</code> when using the Python interface	ON (non-Windows), OFF (Windows)

For more details on specific options, refer to the comments in the `CMakeLists.txt` file.

Interfaces installation

For the installation of Python/MATLAB/Octave interfaces, please refer to the [Interfaces](#) page.

Windows 10+ (WSL)

Prerequisites

- Install [Ubuntu on WSL](#) using the [Microsoft Store](#).
- Start Ubuntu, the shell should pop up.
- Update apt

```
apt-get update
```

- Install `cmake`, `build-essentials`, `pip`, `virtualenv`

```
apt-get install cmake build-essential python3-pip python3-virtualenv
```

Clone, Build and Install acados

- Navigate to the directory where you would like to install acados. For example

```
cd /mnt/c/Users/Documents/
```

- Follow the [Linux/Mac workflow above and install acados using cmake](#).

Interfaces installation

For the installation of Python/MATLAB/Octave interfaces, please refer to the [Interfaces](#) page.

Windows (native, for use with MATLAB)

Prerequisites

You should have the following software installed on your machine.

- Recent MATLAB version
- CMake for Windows
- [Windows Git Client](#)

Clone acados

Clone `acados` and its submodules by running the following from your Git shell:

```
git clone https://github.com/acados/acados.git
cd acados
git submodule update --recursive --init
```

Prepare acados build (minGW)

- Install mingw from MATLAB add-ons manager.
- Add the following paths to your environment variable `%PATH%` using the Windows GUI. To open the GUI press Windows key and type "env".

- The path to the `cmake.exe` file. The default location is `C:\Program Files\CMake\bin`.
- The path to the MATLAB installation of `mingw32-make.exe` file. The default location is `C:\ProgramData\MATLAB\SupportPackages\R2018a\3P\instrset\mingw_w64\instrset\bin`.

- You can check whether the modification to `%PATH%` variable is in effect by executing in cmd `echo %PATH%` or in PowerShell `echo $env:path`.

Automatic build of acados (minGW)

Run the following in MATLAB in the folder `<acados_root_folder>/interfaces/acados_matlab_octave`:

```
acados_install_windows()
```

This will build acados with the standard options and install the external dependencies; casadi and terra renderer

The `acados_install_windows(CMakeConfigString)` script can take an optional argument:

- CMake configuration string: Configuration options for CMake. The default is `-DBUILD_SHARED_LIBS=OFF -DACADOS_WITH_OSQP=OFF`

Using the Python interface simultaneously

The MATLAB interface uses static libraries on Windows and the Python interface has to be used with shared libs. If one wants to use both interfaces without recompiling acados in between, the following workflow was tested.

In MATLAB, run the following script

```
acados_install_shared_and_static_windows()
```

This internally uses `acados_install_windows`. It compiles first the shared libraries and moves them into `<acados_root>/bin`. Then, it compiles the static libraries, they are located in `<acados_root>/lib`.

When using acados in Python, the following options need to be set when creating an OCP solver.

```
cmake_builder = ocp_get_default_cmake_builder()
cmake_builder.generator = 'MinGW Makefiles'
ocp.acados_lib_path = os.path.join(get_acados_path(), 'bin').replace(os.sep, '/')
ocp_solver = AcadosOcpSolver(ocp, cmake_builder=cmake_builder)
```

And similarly for an `AcadosSimSolver`.

The MATLAB interface can be used as in the standard workflow.

Build acados manually (minGW)

If the automated install procedure does not work acados can be built manually using these steps.

Run the following from a powershell in the `<acados_root_folder>`:

```
$ACADOS_INSTALL_DIR=$(pwd)
mkdir -p build
cd build
```

Configure the `cmake` command if you want to use other external QP solvers or change the `HIPM` and `BLASFEO` targets.

```
cmake.exe -G "MinGW Makefiles" -DACADOS_INSTALL_DIR="$ACADOS_INSTALL_DIR" -DBUILD_SHARED_LIBS=OFF -DACADOS_WITH_OSQP=ON ..
# useful options to add above:
# -DACADOS_WITH_QPOASES=ON/OFF -DACADOS_WITH_OSQP=ON/OFF -DACADOS_WITH_QPDUNES=ON/OFF ..
# -DBLASFEO_TARGET=GENERIC -DHPIM_TARGET=GENERIC
# NOTE: check the output of cmake: -- Installation directory: should be <acados_root_folder>,
#       if this is not the case, set -DACADOS_INSTALL_DIR=<acados_root_folder> explicitly above.
```

In a powershell, navigate to the folder `<acados_root_folder>/build` and execute

```
mingw32-make.exe -j4
mingw32-make.exe install
```

Try a MATLAB example

- Open MATLAB
- select MinGW compiler using `mex`
- go to `<acados_root_folder>/examples/acados_matlab_octave`
- run `acados_env_variables_windows`
- go to the `getting_started` subfolder
- run `minimal_example_ocp`

Workflow with Microsoft Visual C Compiler (MSVC)

Note: this workflow is preliminary and not thoroughly tested. (Tested once with MSVC 2017 and MSVC 2019 on May 2021)

- clone acados (see above)
- use the `Developer Command Prompt for VS`, navigate to `<acados_root_folder>/build` and run

```
cmake -G "Visual Studio 15 2017 Win64" -DBLASFEO_TARGET=GENERIC -DACADOS_INSTALL_DIR=.. -DBUILD_SHARED_LIBS=OFF ..
# respectively for MSVC 2019
# cmake -G "Visual Studio 16 2019" -DBLASFEO_TARGET=GENERIC -DACADOS_INSTALL_DIR=.. -DBUILD_SHARED_LIBS=OFF ..
cmake --build . -j8 --target INSTALL --config Release
```

- In MATLAB, run `mex -setup C` and select the same `MSVC` version.
- Try a MATLAB example (see above).

Notes for MacOS

Installing acados with OpenMP on macOS

The standard `clang` compiler on macOS does not support OpenMP. Here, we solve this problem by installing `gcc` and make it the default compiler for `acados`. Using OpenMP allows for parallelization, which can significantly speed up computations, either within one OCP solution or for solving batches of OCPs or simulation problems.

1. Make `gcc` your default compiler for acados

First, install `gcc` using Homebrew. If you do not have Homebrew installed, you can find instructions [here](#).

```
# Install gcc via Homebrew
brew install gcc
```

Next we link the `gcc` compiler to `cc` and `c++` so that acados uses `gcc` instead of the default `clang` compiler.

```
brew link gcc
# you might need to force it
# brew link gcc --overwrite --force
```

2. Set environment variables

Set in your shell configuration file `.bashrc`, `.zshrc`, etc. the following environment variables to point to the installed `gcc` compiler. Adjust the version number (`gcc-15` and `g++-15`) if a different version is installed. You also might need to check whether the path `/opt/homebrew/bin/` is correct for your Homebrew installation:

```
export CC="/opt/homebrew/bin/gcc-15"
export CXX="/opt/homebrew/bin/g++-15"
```

Deprecated: Make build system

NOTE: This build system is not actively tested and might be removed in the future! It is strongly recommended to use the `CMake` build system.

Set the `BLASFEO_TARGET` in `<acados_root_folder>/Makefile.rule`. Since some `C` examples use `qpOASES`, also set `ACADOS_WITH_QPOASES = 1` in `<acados_root_folder>/Makefile.rule`. Install `acados` as follows:

```
make shared_library
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:<path_to_acados_folder>/lib
make examples_c
make run_examples_c
```

NOTE: On MacOS `DYLD_LIBRARY_PATH` should be used instead of `LD_LIBRARY_PATH`.